

PPAD (PPGCC)

Programação em MPI

Relatório — Opção 2

Integrante: Gian Franco Joel Condori Luna

Entrega: 05/12

1 Introdução

O objetivo desta atividade foi investigar estratégias de paralelização utilizando o modelo de programação distribuída **MPI (Message Passing Interface)**, aplicadas ao problema clássico de autômatos celulares conhecido como *Game of Life*, proposto originalmente por John Conway.

2 Descrição do código-fonte

O projeto desenvolvido contém os seguintes componentes principais:

- `serial_gol.c`: implementação sequencial do Game of Life com medição de tempo baseada em `clock_gettime()`;
- `mpi_gol.c`: versão paralela utilizando MPI, com troca de fronteiras entre processos;
- `Makefile`: arquivo para compilação automatizada das duas versões;
- `run_experiments.sh`: script para execução repetida variando o número de processos MPI.

Estratégia do algoritmo

Cada versão (sequencial e paralela) opera sobre duas matrizes: `grid` e `newgrid`, representando a geração atual e a próxima. A inicialização segue um padrão pseudo-aleatório controlado pelo valor fixo:

```
#define SRAND_VALUE 1985
```

O valor final esperado de células vivas na última geração é especificado no enunciado do exercício, o que permite validar a implementação.

Na versão MPI, a grade bidimensional é particionada por linhas entre os processos. A cada iteração ocorre:

1. cálculo independente das linhas internas de cada processo;
2. troca de duas linhas de halo (superior e inferior) entre processos vizinhos;
3. sincronização implícita devido às operações de comunicação;
4. atualização global após cada geração.

3 Plataforma de execução

Os experimentos foram realizados em um computador pessoal com as seguintes características:

- **Processador:** Intel Core i7-11800H (8 núcleos físicos / 16 threads)
- **Memória RAM:** 16 GB DDR4

- **Sistema operacional:** Ubuntu 24.04 LTS (64 bits)
- **Implementação MPI:** MPICH
- **Compiladores:** gcc 13.2, mpicc

Ressalta-se que se trata de um ambiente de nó único, sem interconexão de rede dedicada entre múltiplas máquinas, fator determinante para interpretação dos resultados.

4 Execução e parâmetros experimentais

Para garantir que o ambiente estivesse corretamente configurado, realizou-se inicialmente a instalação da implementação MPICH do MPI. Em sistemas Ubuntu, caso o ambiente não possua MPI previamente instalado, o procedimento recomendado é:

```
sudo apt update
sudo apt install mpich -y
```

Após a instalação, a compilação dos programas (`serial_gol` e `mpi_gol`) é realizada simplesmente com:

```
make
```

Com os executáveis gerados, preparou-se o script de automação `run_experiments.sh`. Para permitir sua execução, concederam-se permissões ao arquivo utilizando:

```
chmod +x run_experiments.sh
```

A coleta dos resultados experimentais foi então realizada executando:

```
./run_experiments.sh
```

Esse script automatiza a repetição dos testes para diferentes valores de processos MPI (`np`), registrando:

- tempo total de execução (*wall-clock*);
- tempo gasto exclusivamente no laço principal de iterações;
- número final de células vivas, para fins de validação.

Ao final da execução, o script gera automaticamente um arquivo CSV contendo todos os resultados coletados. Para os parâmetros utilizados nesta atividade, o arquivo produzido foi:

```
results_dim2048_gens2000.csv
```

Esse arquivo serviu como base para a análise apresentada nas seções seguintes.

5 Resultados

A Tabela 1 apresenta os tempos totais obtidos para cada configuração de número de processos MPI na execução do Game of Life com tabuleiro 2048×2048 e 2000 gerações. Cada configuração foi executada duas vezes para aumentar a confiabilidade das medições.

Tabela 1: Resultados experimentais para o tabuleiro 2048×2048 com 2000 gerações. Cada configuração de processos MPI foi executada duas vezes.

np	Execução	Tempo total (s)	Tempo do loop (s)	Células vivas finais
1	1	48.752831	48.751561	146951
1	2	47.495253	47.493658	146951
2	1	53.109320	53.109319	146951
2	2	54.839895	54.839894	146951
4	1	69.566226	69.566226	146951
4	2	71.383794	71.383793	146951
8	1	133.314710	133.314710	146951
8	2	133.657994	133.657993	146951

Em todas as execuções, o número final de células vivas foi:

$$\text{final_live} = 146951,$$

exatamente o valor esperado segundo o enunciado do problema. Isto valida a corretude tanto da versão sequencial quanto da versão paralela do código desenvolvido.

6 Análise crítica dos resultados

Os resultados obtidos mostram um comportamento importante: o aumento no número de processos MPI não reduziu o tempo de execução. Na verdade, observou-se um aumento significativo do tempo total, especialmente a partir de $np = 4$.

Esse comportamento, ainda que contraintuitivo, é coerente com a natureza do algoritmo e com o ambiente experimental utilizado.

Sobrecarga de comunicação

Em cada geração, cada processo realiza quatro operações de comunicação ponto a ponto:

$$2 \text{ envios} + 2 \text{ recebimentos} = 4 \text{ operações por iteração.}$$

Para 2000 gerações, isso resulta em:

$$8000 \text{ operações de comunicação por processo.}$$

Como os experimentos ocorreram em um único nó, essas trocas não ocorrem em uma rede de alta performance, mas por meio de cópias de memória via kernel, o que introduz latências perceptíveis. Assim, a comunicação passa a dominar o custo total.

Desequilíbrio entre computação e comunicação

Com o aumento de np , cada processo recebe uma fatia menor do tabuleiro. Isso reduz o custo computacional local, porém:

O custo de comunicação permanece aproximadamente constante.

A razão *computação/comunicação* piora, levando à situação:

$$T_{\text{par}}(np) > T_{\text{seq}}, \quad \forall np > 1.$$

Ou seja, o problema é dividido em partes demasiado pequenas para justificar a comunicação introduzida pelo MPI.

Sincronização e dependências

As trocas de fronteira atuam como barreiras implícitas: todos os processos devem concluir sua computação local antes de avançar. Pequenas variações no tempo de execução entre processos amplificam o custo de sincronização, degradando ainda mais o desempenho.

7 Considerações finais

Apesar da ausência de speedup, os experimentos foram bem-sucedidos nos seguintes aspectos:

- validação da corretude da implementação paralela;
- reprodução exata do número final de células vivas esperado para o problema;
- observação empírica de como algoritmos com comunicação intensa sofrem penalidades em MPI;
- compreensão detalhada da relação entre computação local, comunicação e sincronização;
- demonstração prática de que paralelização não implica necessariamente aceleração.

De forma geral, os resultados evidenciam que o paralelismo distribuído deve ser aplicado com cautela, considerando não apenas o tamanho do problema, mas também a granularidade da decomposição e as características do ambiente de execução. Em arquiteturas de nó único, algoritmos com comunicação fina, como o *Game of Life*, raramente apresentam ganhos quando paralelizados com MPI.

Referências

- [1] Message Passing Interface Forum. (2023). *MPI Standard — Version 4.1*.
- [2] OpenAI. (2025). *ChatGPT (GPT-5) [modelo de linguagem de grande escala]*. Disponível em: <https://chat.openai.com/>